

Project 4 —

Findings from reading the Project 4 brief against the repository, the corrected technical design, and the current implementation board created from it. No repository files, branches, commits, or pushes were made to produce this plan or its board setup.

Repo: [excellentteam-bootcamp-2026/firewall-RomiSinizkey](https://github.com/excellentteam/bootcamp-2026/tree/main/firewall-RomiSinizkey)

Board: Student Firewall Project (#2)

Prior epics: #5, #10, #25

Updated: 2026-08-20

✓ **v4 — planning and board setup completed.** Epic #47 and its six native sub-issues (#41–#46) are created and live on the Student Firewall Project board, matching the corrected design below: a compiled `src/main/migrate.ts` migration runner (not `npm run db:migrate`, which needs the devDependency `drizzle-kit`), explicit `POSTGRES_USER` / `POSTGRES_PASSWORD` / `POSTGRES_DB` → `DB_USER` / `DB_PASSWORD` / `DB_NAME` mapping, safe `.env.*.example` templates with gitignored real files, separate dev/prod Compose files, health checks, and per-environment persistent volumes. Scope is still **backend + PostgreSQL only**; frontend stays deferred, described but not created. 20 ready story points, dependency order preserved end to end.

01 What's actually in the repo

FRONTEND

Absent — out of scope

No frontend directory, package.json, or framework anywhere in this repo. Decided: not implemented this pass.

HEALTH ENDPOINT

Absent

`app.ts` only mounts `/api/firewall`. No `/health` route exists.

DOCKERFILES / COMPOSE

None exist

No Dockerfile, .dockerignore, or docker-compose file anywhere in the tree.

DOCKER ENGINE

Installed, not running

`docker version` failed — Docker Desktop/engine was installed but not started. Now an explicit prerequisite (§04, §07).

MIGRATIONS ON STARTUP

Decided: Docker-only

Automatic only inside the container entrypoint, via a compiled `src/main/migrate.ts` runner — not `drizzle-kit`. Host-machine `npm run db:migrate` stays manual and unchanged.

DB CONNECTION RESILIENCE

Already built

Singleton pool with 5-attempt Stop-and-Wait + exponential backoff (`connection.ts`).

CI WORKFLOW

npm-only

Lint/build/test against a Postgres service container. No image build or push step.

PROJECT BOARD FIELDS

Status · Priority · Points

Priority and Story Points live on the board, not in issue body text — confirmed from #25–#33.

GITHUB BOARD (THIS PASS)

Epic #47 live

Six native sub-issues (#41–#46) created, dependency-ordered, 20 story points — see §07.

02 Answers to your ten questions

In the order you asked them. Updated where a decision has since been made; original evidence kept intact.

Q1 Does this repository currently contain a frontend?

No. `src/` contains only the hexagonal backend (adapters, application, domain, main). There is no `frontend/` or `client/` directory, no second `package.json`, and no framework dependency (React/Vue/Vite/etc.) anywhere in `package.json` or `package-lock.json`.

Q2 If not, is there evidence a frontend is expected from another repo or a future project?

No direct evidence, only an inference. `README.md`, `GAP_REPORT.md`, and `AI_CONVERSATION_REPORT.md` never mention a frontend, and the adjacent `firewall-chat-exports` folder is just archived AI-conversation exports for PRs #1–#34, not source code. The PDF itself is worded as if a frontend already exists (Task 1: "Define a Dockerfile for the front end"; Task 4: Compose should hold "3 images now (backend, front end and Postgres)") — brief language, not repository evidence, and it doesn't say where that frontend lives.

Q3 Should frontend Docker tasks be blocked until the frontend location/scope is clarified?

Decided: yes — deferred, not implemented. This project implements only the components that currently exist: the backend and PostgreSQL. No frontend is invented. Tasks 1, 2, and 4 stay described in the Epic (#47) and in the internal placeholder 4.6 (no GitHub issue created for it) as a named, blocked placeholder — not silently dropped, not guessed at — and pick back up once a frontend is actually provided.

Q4 Does the application currently expose a health endpoint suitable for Docker health checks?

No. `app.ts` routes are `/api/firewall/*` plus a 404/error handler — nothing else. This needs a small new route before a Dockerfile `HEALTHCHECK` or a Compose `service_healthy` condition can target the backend itself. Recommend a cheap liveness check (`GET /health` → `200 {"status":"ok"}`) that does *not* query Postgres — a DB-backed health check would make the container flap unhealthy during Postgres's own restarts, double-counting a failure the DB healthcheck and the app's own connection retry already handle.

Q5 How should migrations run when the containers start?

Decided: automatically, but only inside the Docker container startup flow — via a compiled runner, not `npm run db:migrate`.

`npm run db:migrate` shells out to `drizzle-kit`, which is a devDependency and does not exist in the production image, so the container entrypoint cannot call it. Instead, both the dev and prod backend entrypoints run a small application migration runner (`src/main/migrate.ts`, compiled to `dist/main/migrate.js`) built on `drizzle-orm/node-postgres/migrator`, reusing the same `pg` / `drizzle-orm` dependencies already in the image — before `node dist/main/server.js` starts. A fresh `postgres` container has zero schema on first boot, unlike Project 3's local Postgres, which a developer sets up and migrates once by hand. The host-machine workflow (`npm run dev`, or a manual `npm run db:migrate` outside a container) is untouched and still uses `drizzle-kit` directly, so the Project 3 Epic's (#25) assumption — *"Migrations run through an explicit Drizzle command, not automatically on application startup"* — still holds outside Docker. The automation is additive and scoped to containers, not a reversal of that assumption.

Q6 Which environment variables are required inside Docker?

Exactly the 8 keys `env.ts` already validates: `ENV`, `PORT`, `DB_CONNECTION_INTERVAL`, `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`. Nothing new needs inventing on the backend side. Two things change only in a Docker context: `DB_HOST` becomes the Postgres **service name** (see Q9), and `TEST_DB_*` vars stay out of the Docker env files entirely — they belong to the host-machine Vitest integration suite, not the running container. No frontend variables, since no frontend is implemented this pass.

Q7 Should the backend use separate development and production Docker stages?

Yes. A multi-stage Dockerfile: a `build` stage running `npm ci && npm run build`, a slim `production` stage that installs prod-only deps and copies just `dist/` (plus `drizzle/` for migrations) onto a non-root user, and a `development` stage that keeps devDependencies and runs `ts-node-dev --respawn` against a bind-mounted `src/` for hot reload. This is what makes `docker-compose.dev.yml` vs `docker-compose.prod.yml` mean something different at the image level, not just a different env file.

Q8 Should PostgreSQL use a named persistent volume?

Decided: yes, and a separate one per environment. `postgres_data_dev` for `docker-compose.dev.yml`, `postgres_data_prod` for `docker-compose.prod.yml`, each mounted at `/var/lib/postgresql/data`. Data survives `docker compose down` / container recreation within an environment and only disappears on an explicit `-v` teardown; a local prod-mode test run can never share state with dev data.

Q9 How should Compose guarantee the backend waits for a healthy Postgres? Is `depends_on` alone sufficient?

No — plain `depends_on` is not sufficient, and this is now the confirmed design in both compose files.

Compose's short-form `depends_on: [postgres]` only sequences **container start order**: it waits for the Postgres container's process to launch, not for PostgreSQL to finish `initdb` and start accepting connections. Those are different moments in time, sometimes seconds apart. Starting the backend right after "postgres started" can still hit connection-refused.

The fix, used in both `docker-compose.dev.yml` and `docker-compose.prod.yml`:

- `postgres` service: `healthcheck: { test: ["CMD-SHELL", "pg_isready -U $POSTGRES_USER -d $POSTGRES_DB"] }`
- `backend` service: `depends_on: { postgres: { condition: service_healthy } }`

That makes Compose *readiness*-gated, not just *order*-gated. The app's own 5-attempt Stop-and-Wait retry in `connection.ts` is then defense-in-depth for slower restarts mid-lifecycle, not the only thing standing between the backend and a cold-start crash loop.

Q10 The PDF says `env.dev` but `.env.prod` — typo or intentional?

Decided: typo. Using `.env.dev`, `.env.prod`, `.env.example` — dot-prefixed throughout. Task 5 lists the three files back to back with no explanation for why one would drop the leading dot while its siblings keep it, and a visible `env.dev` sitting next to hidden `.env.prod` / `.env.example` would be an inconsistent convention with no functional reason — dotenv, Docker's `env_file`, and this repo's existing `.env.example` / `.gitignore` entry all assume the dotted family. The literal `env.dev` spelling from the PDF is still recorded in issue 4.3 / #43's body for grader visibility — not silently dropped, just not used as the actual filename.

03 Ambiguities in the brief — now resolved

Migrations-on-startup Decided

Epic #25 recorded migrations as manual-only, by design.

Decision: automatic only inside the Docker entrypoint (dev + prod). Non-Docker local workflow stays manual — #25's assumption still holds outside containers.

Env filenames Decided

PDF literally lists `env.dev` next to `.env.prod` and `.env.example`.

Decision: `.env.dev`, `.env`

Frontend assumed to already exist

Decided

Tasks 1, 2 and 4 are written as if a frontend is already in hand. It isn't — in this repo or any repo/doc this session can see.

Decision: implement only backend + PostgreSQL this pass. Frontend work stays a described, blocked placeholder (internal 4.6, no GitHub issue) until one is provided.

One compose file vs. two, later split

Decided

PDF's Task 3 → Task 6 progression reads as one generic file first, then a dev/prod split.

Decision: go straight to two standalone files,

`docker-compose.dev.yml`

and

`docker-compose.prod.yml`

— no throwaway generic file. Task 3's backend+Postgres wiring is delivered directly as the dev file (issue 4.4 / #44).

04 Epic #47 — created

TRACKING

DOCKER

#47 · In Progress

Project 4: Dockerize the Backend and PostgreSQL (Frontend Deferred)

Mirrors the shape of #5 / #10 / #25. Created on the Student Firewall Project board, labels `tracking` + `docker`, Status **In Progress**. No Story Points on the Epic itself, matching the #25 convention.

WHAT THIS COVERS

- Containerize the Express/TypeScript backend with dev and prod stages, and PostgreSQL, then connect them through two environment-specific Compose files with real startup ordering (health-gated, not just start-order-gated).
- Add a liveness endpoint the backend image can use for its own `HEALTHCHECK`.
- Run schema migrations automatically only inside the Docker container startup flow, via a compiled application migration runner (not `drizzle-kit`, which isn't in the production image) against a fresh, empty Postgres. Local, non-Docker migrations remain manual and keep using `npm run db:migrate`.
- Backend env files for dev/prod/example, delivered as committed safe templates plus locally-created real files so a fresh clone stays reproducible, and `docker-compose.dev.yml` / `docker-compose.prod.yml` at the repo root.

PRECONDITION (CHECKLIST ITEM, NOT AN ISSUE)

- Install Docker Desktop/Engine locally, then **start** it — installed is not the same as running. This environment hit exactly that gap: Docker was installed but `docker version` failed because Desktop wasn't started. Verify with `docker version` and `docker compose version` before continuing. No repository artifact results from this step, so per your decision it's tracked here rather than as its own issue — a deliberate break from the #26 precedent ("*[Setup] Install PostgreSQL and configure DBeaver*" did get its own issue in Project 3, also with zero repo artifacts).

DEFERRED (NO GITHUB ISSUE CREATED)

- Frontend Dockerfile, image verification, and its Compose slot — out of scope for this pass by decision. Documented here as scope, not filed as an issue (internal reference 4.6); picked up once a frontend is actually provided, not before.

Required engineering total (backend + Postgres only): **20 Story Points** — created and on the board

Frontend: **deferred — not scheduled**, ~9 pts once provided (estimate)

05 Created GitHub issues

Six native sub-issues of Epic #47, created in dependency order, plus one described-but-not-created frontend placeholder. Each card below shows the internal planning label next to the real GitHub issue number (e.g. `4.1 / #41`).

4.1 / #41 **[API] Add a health check endpoint**

In Progress

High

OBJECTIVE

Add `GET /health` returning `200 {"status":"ok"}`, mounted outside `/api/firewall`, with no database dependency.

ACCEPTANCE CRITERIA

- Route responds 200 with no Postgres connection in the request path (pure liveness, not readiness).
- Unit/HTTP test added alongside the existing Supertest coverage in `tests/integration/http`.
- Documented in README's route table next to the existing `/api/firewall/*` entries.

Board status

In Progress

Depends on

none

Suggested branch

feature/health-endpoint

Story points

2

Labels

enhancement

PDF task

none — identified gap (Q4)

4.2 / #42 **[Docker] Add backend Dockerfile with dev/prod stages**

Backlog

High

OBJECTIVE

Multi-stage Dockerfile (`build` → `production`, plus a `development` target), a `.dockerignore`, and a compiled application migration runner so both runtime stages can migrate without depending on `drizzle-kit`.

ACCEPTANCE CRITERIA

- `production` stage: prod-only deps (`npm ci --omit=dev` — `drizzle-kit` is *not* present), copies `dist/` + `drizzle/` (the SQL migration files, still needed as input), runs as non-root.
- `development` stage: keeps devDependencies, runs `ts-node-dev --respawn` against a bind-mounted `src/`.
- New: `src/main/migrate.ts` — a migration runner built on `drizzle-orm/node-postgres/migrator`'s `migrate()`, reusing the existing production `pg / drizzle-orm` dependencies. It opens its own short-lived `pg Pool`, runs `migrate(db, { migrationsFolder: "./drizzle" })`, and always closes that pool afterward (`pool.end()`) — it must

Board status

Backlog

Depends on

4.1 / #41

Suggested branch

feature/backend-dockerfile

Story points

5

Labels

docker

PDF task

Task 3 (backend Dockerfile)

not leak into the app's long-lived Singleton pool in `connection.ts`.

- Production entrypoint runs `node dist/main/migrate.js && node dist/main/server.js` — **not** `npm run db:migrate`, since `drizzle-kit` is a devDependency absent from the production image. The `&&` chain means a failed migration (non-zero exit) stops the container before the server ever starts — migration failure must fail startup, not continue against a mismatched schema.
- Development stage runs the same `src/main/migrate.ts` (via `ts-node`) before `ts-node-dev` starts — one migration code path for both Docker environments, neither depending on `drizzle-kit`.
- `HEALTHCHECK` instruction targets the new `/health` route from 4.1 / #41.
- `.dockerignore` excludes `node_modules`, `dist`, `.env*`, `.git`, `tests` — keeps `drizzle/` (needed at runtime as migration input, independent of `drizzle-kit`).
- The host-machine `npm run db:migrate` / `npm run dev` workflow (no container) is untouched and still uses `drizzle-kit` directly.

4.3 / #43 [Config] Add backend dev/prod/example env files

Backlog Medium

OBJECTIVE

Create safe, committed `.env.dev.example` / `.env.prod.example` templates plus the real, gitignored `.env.dev` / `.env.prod` files developers generate from them, and refresh `.env.example`, matching the 8-key schema in `env.ts`.

ACCEPTANCE CRITERIA

- Commit `.env.dev.example` and `.env.prod.example` — placeholder values only (e.g. `DB_PASSWORD=change_me`), same pattern as the existing `.env.example`. These are what make the setup reproducible on a fresh clone.
- `.env.dev` and `.env.prod` — the real files Docker actually reads via `env_file` — are created locally per developer with `cp .env.dev.example .env.dev` and `cp .env.prod.example .env.prod`, then filled in with real local/prod values. Both are added to `.gitignore` alongside the existing `.env` entry; a real production password is never committed.
- README/setup docs spell out the two `cp` commands above as a required one-time step per clone, before the first `docker compose ... up`.

Board status	Backlog
Depends on	4.2 / #42
Suggested branch	feature/backend-env-files
Story points	2
Labels	configuration
PDF task	Task 5

- `.env.dev.example` / `.env.dev` set `DB_HOST` to the Compose Postgres service name, not `localhost` (see 4.4 / #44). `.env.example` (host-machine, non-Docker) is untouched and keeps `DB_HOST=localhost`.
- This satisfies PDF Task 5's three named files exactly (`.env.dev`, `.env.prod`, `.env.example`) — the two new `.example` templates are what make those three reproducible on a fresh clone, not a fourth/fifth deliverable competing with the brief.
- Issue body notes the PDF literally says `env.dev` (no dot); filename used is `.env.dev` — decided, see Q10.

4.4 / #44 [Docker] Add docker-compose.dev.yml (backend + PostgreSQL)

Backlog

High

OBJECTIVE

Root-level `docker-compose.dev.yml` wiring backend + Postgres for local development — delivers Task 3's backend+Postgres wiring directly as the dev environment file, with no separate generic compose file.

ACCEPTANCE CRITERIA

- `postgres` service: named volume `postgres_data_dev:/var/lib/postgresql/data`, `healthcheck` via `pg_isready`, port **5432 published to the host** for DBeaver access.
- `postgres` service `environment:` maps Compose-substituted values from `.env.dev` onto the variables the official image expects — `POSTGRES_USER: ${DB_USER}`, `POSTGRES_PASSWORD: ${DB_PASSWORD}`, `POSTGRES_DB: ${DB_NAME}` — one source of truth, no duplicated credentials.
- `backend` service: built from the Dockerfile's `development` stage, `env_file: .env.dev`, source bind-mounted for hot reload.
- `depends_on: postgres: condition: service_healthy` — not plain `depends_on` (see Q9).
- `DB_HOST` in `.env.dev` is the Postgres **service name**, never `localhost` — inside a Compose network, `localhost` resolves to the backend container itself.
- Documented run command, so the `${...}` substitutions above actually resolve: `docker compose --env-file .env.dev -f docker-compose.dev.yml up --build`. The backend still gets its own variables independently via `env_file: .env.dev` on the service — both mechanisms read the same file for two different purposes (top-level substitution vs. injecting env vars into a container).

Board status

Backlog

Depends on

4.2 / #42, 4.3 / #43

Suggested branch

feature/compose-dev

Story points

5

Labels

docker, database

PDF task

Task 3, and starts Task 6

OBJECTIVE

Root-level `docker-compose.prod.yml`, reusing 4.4 / #44's proven service topology with production-specific settings. Completes Task 6.

ACCEPTANCE CRITERIA

- `postgres` service: separate named volume `postgres_data_prod:/var/lib/postgresql/data`, same `pg_isready` healthcheck, port **not published** to the host.
- `postgres` service `environment:` maps the same three keys as 4.4 / #44, this time from `.env.prod`: `POSTGRES_USER: ${DB_USER}`, `POSTGRES_PASSWORD: ${DB_PASSWORD}`, `POSTGRES_DB: ${DB_NAME}`.
- `backend` service: built from the Dockerfile's `production` stage, `env_file: .env.prod`, no source bind-mount.
- Same `service_healthy` condition as 4.4 / #44.
- Documented run command: `docker compose --env-file .env.prod -f docker-compose.prod.yml up --build`.
- Two standalone files, not a base + override pair — matches the decided approach in §03.

Board status	Backlog
Depends on	4.4 / #44
Suggested branch	feature/compose-prod
Story points	3
Labels	docker
PDF task	Task 6

OBJECTIVE (ONCE A FRONTEND IS PROVIDED)

Frontend Dockerfile + `.dockerignore`, verified standalone image, then added as a third Compose service alongside backend and Postgres in both dev and prod files, with the backend's URL injected via a frontend env var.

WHY THIS ISN'T A REAL ISSUE YET

- Decided: only the backend and PostgreSQL are implemented this pass. No frontend code, framework, port, or build tool exists to base a Dockerfile on.
- Kept here as a named, described placeholder so scope stays visible in the Epic — not silently dropped.

Board status	Not on board
Depends on	frontend provided, 4.5 / #45
Suggested branch	—
Story points	~9 (est.)
Labels	docker, frontend*
PDF task	Tasks 1, 2, 4

OBJECTIVE

Board status	Backlog
--------------	---------

Document prerequisites, dev/prod run commands, env file setup, migration behavior, and volume reset steps; run a full smoke test of both compose files. Mirrors the rigor of #32 for Project 3.

ACCEPTANCE CRITERIA

- README (or a new `DOCKER.md`) states Docker Desktop/engine must be **installed and running** before any `docker` command — call out that installed-but-not-running is a real, easy-to-hit gap (encountered directly during this planning pass).
- Documents the one-time setup step per clone: `cp .env.dev.example .env.dev` and `cp .env.prod.example .env.prod`, then fill in real local/prod values (issue 4.3 / #43) — required before the first `docker compose up`.
- Covers: `docker compose --env-file .env.dev -f docker-compose.dev.yml up --build` / `docker compose --env-file .env.prod -f docker-compose.prod.yml up --build`, how migrations run automatically only inside containers via the compiled `migrate.js` runner (issue 4.2 / #42), how to reset each environment's volume independently, and DB Beaver connection steps for dev (prod intentionally has no host DB access).
- Smoke test executed and results recorded: fresh `up` reaches a healthy backend against an empty Postgres with migrations applied automatically via `dist/main/migrate.js`; data survives a container restart; data is gone only after an explicit volume teardown; prod's Postgres port is confirmed unreachable from the host; a deliberately-broken migration is confirmed to stop the container before the server starts.

Depends on	4.5 / #45
Suggested branch	feature/docker-docs-smoke-test
Story points	3
Labels	documentation, docker
PDF task	Task 6 (wrap-up)

06 Style vs. Projects 1–3

CONVENTION	PROJECTS 1–3 (OBSERVED IN #26–#33)	THIS PROPOSAL
Issue body	Terse: <code>Depends on</code> / <code>Suggested branch</code> / one-paragraph <code>Objective</code> . No explicit acceptance-criteria list.	Adds a short AC bullet list — recommend trimming to match precedent when actually filing, keeping the detailed AC here only as working notes.
Priority / Story Points	Never in the markdown body. Points appear once, aggregated, on the Epic (" <i>Required engineering total: 25 Story Points</i> "). Priority isn't in any body at all.	Matches this — Priority and Points shown here are board-field values to set after creation, not text to paste into the issue.
Local/no-code setup steps	Get their own issue anyway (#26, " <i>Local machine setup — no code, no branch</i> ").	Deviates by decision: Docker install/start is an Epic checklist line, not an issue. A conscious, confirmed break from precedent (see §08).
Optional/non-blocking work	Explicit <code>[Optional]</code> title prefix + a line stating it doesn't block the Epic and carries no Milestone (#33).	Frontend issue uses the same pattern — <code>[Blocked]</code> prefix, explicit non-blocking note — once it's actually created.
Assumptions the brief doesn't state	Epic body carries a dedicated " <i>Implementation decisions — assumptions requiring instructor confirmation</i> " section (#25).	Same pattern recommended for the Project 4 Epic — now populated with confirmed decisions rather than open questions (§08).
Labels	New label introduced per project's new concern (<code>database</code> for Project 3).	Introduce <code>docker</code> the same way; add <code>frontend</code> only once 4.6 is unblocked.

07 Current implementation board

Verified state of Epic #47 and its six sub-issues on the Student Firewall Project board, read back from GitHub after creation — not the original proposal.

#	ISSUE	STATUS	PRIORITY	POINTS	DEPENDS ON	BRANCH	PDF TASK
#47	Epic – Dockerize the Backend and PostgreSQL	In Progress	High	–	–	–	tracking
#41	[API] Add a health check endpoint	In Progress	High	2	–	feature/health-endpoint	gap (Q4)
#42	[Docker] Add backend Dockerfile (dev/prod stages)	Back Log	High	5	#41	feature/backend-dockerfile	Task 3
#43	[Config] Add backend dev/prod/example env files	Back Log	Medium	2	#42	feature/backend-env-files	Task 5
#44	[Docker] Add docker-compose.dev.yml	Back Log	High	5	#42, #43	feature/compose-dev	Task 3 / starts 6
#45	[Docker] Add docker-compose.prod.yml	Back Log	High	3	#44	feature/compose-prod	Task 6
#46	[Docs] Document and smoke-test the Docker setup	Back Log	Medium	3	#45	feature/docker-docs-smoke-test	Task 6 (wrap-up)
–	[Docker][Blocked] Dockerize the frontend	Not created	–	~9	frontend provided, #45	–	Tasks 1, 2, 4

Dependency order preserved end to end: #41 → #42 → #43 → #44 → #45 → #46. All six confirmed as native sub-issues of Epic #47 by reading `subIssues` / `parent` back from GitHub. Docker installation/start stays an Epic checklist line, not a numbered row. The frontend row has no GitHub issue and stays unscheduled until one is provided.

What Story Points mean here: an estimate of effort and complexity, not calendar time or hours. **2** — a small, narrowly-scoped task (e.g. one new route). **3** — a medium task with a few moving parts (e.g. a documentation pass plus a smoke test). **5** — a larger or more complex task touching multiple files or requiring more design and verification surface (e.g. a multi-stage Dockerfile with a new migration runner). The 20-point total is the sum across #41–#46 only; the Epic itself carries no points, matching the Project 3 epic's (#25) convention.

08 Decisions Log

All six previously-open items are resolved. Recorded here for traceability, not as open questions.

✓ **Scope: implement only what currently exists.**

Backend + PostgreSQL only. No frontend is invented. Frontend Docker tasks stay as described, blocked placeholder (internal 4.6, no GitHub issue) until a frontend is provided.

✓ **Migrations run automatically only inside the Docker container startup flow.**

Both dev and prod entrypoints run a compiled application migration runner (`src/main/migrate.ts`, built on `drizzle-orm/node-postgres/migrator`) before the server starts — not `npm run db:migrate`, since that shells out to `drizzle-kit`, a devDependency absent from the production image. A failed migration exits non-zero and stops the container before the server starts. The host-machine workflow stays manual and keeps using `drizzle-kit` directly — Project 3 Epic #25's assumption is preserved outside containers.

✓ **Env filenames: `.env.dev`, `.env.prod`, `.env.example`.**

Dot-prefixed throughout, consistent with this repo's existing convention. Delivered via committed `.env.dev.example` / `.env.prod.example` templates plus locally-created real files, so the three PDF-named files exist reproducibly on a fresh clone without ever committing real credentials (issue 4.3 / #43). The PDF's literal `env.dev` spelling is recorded in issue 4.3 / #43 for grader visibility, not used as the actual filename.

✓ **Two standalone Compose files: `docker-compose.dev.yml` and `docker-compose.prod.yml`.**

No generic base file. Task 3's backend+Postgres wiring is delivered directly as the dev file (issue 4.4 / #44); the prod file (issue 4.5 / #45) reuses that topology.

✓ **PostgreSQL healthcheck + `depends_on: condition: service_healthy` in both files.**

Plain `depends_on` only waits for container start, not for Postgres to accept connections — see Q9. Both compose files use the health-gated form.

✓ **Separate named Postgres volumes per environment; port published in dev only.**

`postgres_data_dev` / `postgres_data_prod`. Dev publishes 5432 for DBeaver access, matching the existing Project 3 workflow. Prod does not publish it — the database stays off the host network.

✓ **Docker installation stays an Epic checklist item, not a separate issue.**

Confirmed despite the #26 precedent (Project 3 gave local Postgres/DBeaver install its own issue). Extended to cover *starting* Docker Desktop, not just installing it — the actual gap hit when `docker version` failed in this environment.

09 AI collaboration summary

How ChatGPT and Claude were used together to get from the Project 4 brief to a verified board. No raw conversation transcript — see the repository's

`AI_CONVERSATION_REPORT.md` for the full, section-by-section record.

ChatGPT and Claude were used collaboratively, in the following order, to:

- analyze the original Project 4 requirements against Docker/Compose fundamentals;
- inspect the current repository (source tree, `package.json`, CI workflow, README) rather than assume its contents;
- identify that no frontend exists anywhere in this repository or its accompanying documentation;
- design the Docker implementation scope around what actually exists — backend and PostgreSQL — deferring the frontend instead of inventing one;
- detect and correct the production migration problem (`npm run db:migrate` depending on the devDependency `drizzle-kit`, which a production image doesn't include);
- detect and correct the PostgreSQL environment-variable mapping (`DB_*` vs. the official image's `POSTGRES_*` names);
- design safe environment-file handling (committed `.example` templates, gitignored real files);
- divide the corrected design into six small, independently reviewable GitHub issues in dependency order;
- verify the Epic, dependencies, sub-issue links, board statuses, milestone, points, and priorities directly against GitHub after creation, rather than assume the writes succeeded.

Planning and board setup completed — Epic #47 and Issues #41–#46 are created and live on the Student Firewall Project board. No repository files, branches, commits, or pushes were made at any point to produce this plan or its board setup. Issue 4.6 (frontend) remains a described, not-created placeholder until a frontend is provided.